

CAPA DOMAIN

Documentacion tecnica en formato Markdown

1. Proposito

La capa `domain` representa el nucleo conceptual del sistema de autenticacion. Su responsabilidad es modelar los conceptos del negocio, definir contratos que el resto del sistema debe respetar y establecer las excepciones del dominio para expresar fallos funcionales sin depender de Django, JWT, correo o base de datos.

En este proyecto, la capa contiene:

- Entidades de negocio.
- Puertos de salida para persistencia, hashing, notificaciones y tokens.
- Excepciones de dominio.
- Un espacio para value objects.

La capa esta disenada siguiendo un enfoque de arquitectura hexagonal: el dominio conoce abstracciones, mientras que la infraestructura implementa esas abstracciones.

2. Alcance dentro de la arquitectura

2.1 Posicion arquitectonica

Interfaces/API -> Application -> Domain <- Infrastructure

El dominio no expone endpoints, no serializa requests y no contiene detalles ORM. Su objetivo es proteger las reglas del negocio frente al acoplamiento tecnologico.

2.2 Responsabilidades reales en este proyecto

En el estado actual del codigo, la capa `domain` cumple principalmente una funcion de:

- Modelo del negocio.
- Definicion de contratos.
- Lenguaje ubicuo para autenticacion y MFA.
- Delimitacion de errores funcionales.

La mayor parte de la orquestacion y de las reglas operativas estan implementadas en la capa `application`, especialmente en los casos de uso de login, registro y verificacion MFA.

3. Estructura del modulo

```
OAuth2/domain/  
  entities/  
    user.py  
    mfa_challenge.py  
  exceptions/  
    auth_exceptions.py  
  ports/  
    user_repository.py  
    mfa_repository.py  
    password_hasher.py  
    token_provider.py  
    notification_service.py  
  value_objects/  
    email.py
```

4. Modelo del dominio

4.1 Entidad User

La entidad **User** representa al sujeto autenticable del sistema.

Atributos

Campo	Tipo	Significado
id	int None	Identificador persistente del usuario
nombres	str	Nombres del usuario
apellidos	str	Apellidos del usuario
email	str	Identificador principal de login
password_hash	str	Hash de la contraseña
is_active	bool	Habilita o bloquea autenticacion
is_staff	bool	Marca de privilegio administrativo
is_email_verified	bool	Estado de verificacion del correo
mfa_enabled	bool	Activa el segundo factor
last_login_at	datetime None	Ultimo acceso registrado

Reglas implícitas

- Un usuario inactivo no puede autenticarse.

- El email funciona como identidad de negocio.
- La contraseña nunca debe circular en claro dentro de la entidad.
- La activacion de MFA modifica el flujo de login.

4.2 Entidad MFACHallenge

La entidad **MFACHallenge** representa un desafio temporal asociado a un usuario y a un proposito concreto.

Atributos

Campo	Tipo	Significado
<code>id</code>	UUID	Identificador unico del desafio
<code>user_id</code>	int	Usuario propietario del desafio
<code>code_hash</code>	str	Hash del codigo MFA
<code>channel</code>	str	Canal de entrega, actualmente EMAIL
<code>purpose</code>	str	Motivo del desafio, por ejemplo LOGIN
<code>expires_at</code>	datetime	Fecha de expiracion
<code>consumed_at</code>	datetime None	Marca de consumo exitoso
<code>attempts</code>	int	Intentos realizados
<code>max_attempts</code>	int	Maximo de intentos permitidos
<code>created_at</code>	datetime	Fecha de creacion

Reglas implícitas

- Un desafio consumido no puede reutilizarse.
- Un desafio expirado debe ser rechazado.
- Si `attempts >= max_attempts`, el desafio queda invalidado funcionalmente.
- El codigo MFA se valida comparando hashes, no texto plano.

4.3 Value Objects

El directorio `value_objects` contiene `email.py`, pero el archivo esta vacio. Eso indica una intencion de evolucion hacia un modelo de dominio mas rico, donde el email podria validar formato, normalizacion y reglas de identidad como objeto de valor.

5. Contratos del dominio

Los puertos definen dependencias abstractas que el dominio y la aplicacion necesitan para ejecutar los casos de uso sin conocer detalles tecnicos.

5.1 UserRepository

Responsabilidad:

- Persistir usuarios.
- Buscar usuarios por email o id.
- Actualizar el ultimo acceso.

Operaciones:

- `save(user)`
- `find_by_email(email)`
- `find_by_id(user_id)`
- `update_last_login(user_id)`

5.2 MFARepository

Responsabilidad:

- Persistir desafios MFA.
- Recuperar el desafio activo por usuario y proposito.
- Actualizar estado e intentos del desafio.

Operaciones:

- `save(challenge)`
- `find_active_by_user_and_purpose(user_id, purpose)`
- `update(challenge)`

5.3 PasswordHasher

Responsabilidad:

- Generar hash de contraseña o código.
- Verificar coincidencia entre un secreto plano y su hash.

Operaciones:

- `hash(plain_password)`
- `verify(plain_password, hashed_password)`

5.4 TokenProvider

Responsabilidad:

- Generar tokens de acceso.
- Generar tokens de refresh.

Operaciones:

- `generate_access_token(user_id, email, roles)`
- `generate_refresh_token(user_id)`

5.5 NotificationService

Responsabilidad:

- Entregar el código MFA por un canal externo.

Operación:

- `send_mfa_code(to_email, code)`
-

6. Excepciones del dominio

Las excepciones definen el lenguaje de error del negocio.

Excepción	Significado
<code>AuthDomainException</code>	Base del dominio de autenticación
<code>InvalidCredentialsException</code>	Usuario o credencial no válida
<code>InactiveUserException</code>	Usuario deshabilitado
<code>UserAlreadyExistsException</code>	Registro duplicado
<code>UserNotFoundException</code>	Usuario inexistente
<code>MFAChallengeNotFoundException</code>	No existe desafío activo
<code>InvalidOrExpiredMFACodeException</code>	Código inválido, expirado o consumido
<code>MFAAttemptsExceededException</code>	Se superó el número de intentos

Estas excepciones permiten desacoplar la semántica del negocio de los códigos HTTP o de errores del framework.

7. Reglas de negocio identificadas

7.1 Registro

- No se puede registrar un email ya existente.
- La contraseña debe almacenarse hashada.
- Un nuevo usuario inicia activo, no staff, no verificado y sin MFA habilitado.

7.2 Login

- El usuario debe existir.
- La contraseña debe coincidir con el hash almacenado.
- El usuario debe estar activo.
- El último login se actualiza antes de la resolución final del flujo.

- Si `mfa_enabled` es falso, el sistema emite tokens directamente.
- Si `mfa_enabled` es verdadero, el sistema deriva a un segundo paso MFA.

7.3 Verificacion MFA

- El usuario del desafio debe existir.
- Debe existir un desafio activo para el proposito solicitado.
- El desafio no debe estar consumido.
- El desafio no debe estar expirado.
- El numero de intentos no debe exceder el maximo permitido.
- Cada verificacion fallida incrementa `attempts`.
- Una verificacion exitosa marca `consumed_at` y emite tokens.

8. Arquitectura estructural de alto nivel

Este diagrama muestra la posicion de `domain` dentro del sistema y la direccion de dependencias.

flowchart LR

```

API[Interfaces API]
APP[Application Use Cases]
DOM[Domain\nEntities + Ports + Exceptions]
INF[Infrastructure Adapters]
DB[(Database)]
MAIL[Email Service]
JWT[JWT Provider]

```

```

API --> APP
APP --> DOM
INF --> DOM
APP --> INF
INF --> DB
INF --> MAIL
INF --> JWT

```

Lectura del diagrama

- La API invoca casos de uso.
 - La capa application utiliza contratos del dominio.
 - La infraestructura implementa esos contratos.
 - El dominio no depende de la infraestructura.
-

9. UML estructural de bajo nivel

Este diagrama muestra entidades, puertos y sus implementaciones conocidas.

```
classDiagram
    class User {
        +int|None id
        +str nombres
        +str apellidos
        +str email
        +str password_hash
        +bool is_active
        +bool is_staff
        +bool is_email_verified
        +bool mfa_enabled
        +datetime|None last_login_at
    }

    class MFAChallenge {
        +UUID id
        +int user_id
        +str code_hash
        +str channel
        +str purpose
        +datetime expires_at
        +datetime|None consumed_at
        +int attempts
        +int max_attempts
        +datetime created_at
    }

    class UserRepository {
        <<interface>>
        +save(user) User
        +find_by_email(email) User
        +find_by_id(user_id) User
        +update_last_login(user_id) None
    }

    class MFARespository {
        <<interface>>
        +save(challenge) MFAChallenge
        +find_active_by_user_and_purpose(user_id, purpose) MFAChallenge
        +update(challenge) None
    }
```

```

class PasswordHasher {
    <<interface>>
    +hash(plain_password) str
    +verify(plain_password, hashed_password) bool
}

class TokenProvider {
    <<interface>>
    +generate_access_token(user_id, email, roles) str
    +generate_refresh_token(user_id) str
}

class NotificationService {
    <<interface>>
    +send_mfa_code(to_email, code) None
}

class DjangoUserRepository
class DjangoMFARespository
class DjangoPasswordHasher
class JwtTokenProvider
class EmailNotificationService

UserRepository <|.. DjangoUserRepository
MFARespository <|.. DjangoMFARespository
PasswordHasher <|.. DjangoPasswordHasher
TokenProvider <|.. JwtTokenProvider
NotificationService <|.. EmailNotificationService

MFAChallenge --> User : belongs to user_id

```

10. Vista logica de alto nivel

El siguiente diagrama representa el flujo general de autenticacion desde una perspectiva de negocio.

```

sequenceDiagram
    actor Cliente
    participant API as API REST
    participant APP as Application
    participant DOM as Domain Contracts
    participant INF as Infrastructure

    Cliente->>API: Envia credenciales

```

```

API->>APP: Construye DTO y ejecuta caso de uso
APP->>DOM: Aplica reglas y contratos
APP->>INF: Usa repositorios, hashing, notificacion y tokens

alt MFA deshabilitado
    INF-->>APP: Usuario valido + tokens
    APP-->>API: AuthResponseDTO con access y refresh token
else MFA habilitado
    INF-->>APP: Usuario valido + envio de codigo
    APP-->>API: AuthResponseDTO con mfa_required = true
end

```

Interpretacion

- El dominio define el vocabulario y las reglas.
 - La aplicacion coordina.
 - La infraestructura ejecuta efectos externos.
-

11. Vista logica de bajo nivel

11.1 Secuencia de login con MFA habilitado

```

sequenceDiagram
    actor Cliente
    participant LoginUC as LoginUserUseCase
    participant UserRepo as UserRepository
    participant Hasher as PasswordHasher
    participant Notifier as NotificationService
    participant TokenProvider as TokenProvider

    Cliente->>LoginUC: execute(LoginDTO)
    LoginUC->>UserRepo: find_by_email(email)
    UserRepo-->>LoginUC: User | None

    alt Usuario no existe
        LoginUC-->>Cliente: InvalidCredentialsException
    else Usuario existe
        LoginUC->>Hasher: verify(password, user.password_hash)
        Hasher-->>LoginUC: true | false

        alt Password invalida
            LoginUC-->>Cliente: InvalidCredentialsException
        else Usuario inactivo
            LoginUC-->>Cliente: InactiveUserException
        end
    end

```

```

else Usuario activo
    LoginUC->>UserRepo: update_last_login(user.id)

    alt MFA habilitado
        LoginUC->>Notifier: send_mfa_code(user.email, code)
        LoginUC-->>Cliente: AuthResponseDTO(mfa_required=true)
    else MFA deshabilitado
        LoginUC->>TokenProvider: generate_access_token(...)
        LoginUC->>TokenProvider: generate_refresh_token(user.id)
        LoginUC-->>Cliente: AuthResponseDTO(tokens)
    end
end
end
end

```

11.2 Secuencia de verificación MFA

sequenceDiagram

```

actor Cliente
participant VerifyUC as VerifyMFAUseCase
participant UserRepo as UserRepository
participant MFARepo as MFARepository
participant Hasher as PasswordHasher
participant TokenProvider as TokenProvider

Cliente->>VerifyUC: execute(VerifyMFADTO)
VerifyUC->>UserRepo: find_by_email(dto.email)
UserRepo-->>VerifyUC: User | None

alt Usuario no existe
    VerifyUC-->>Cliente: UserNotFoundException
else Usuario existe
    VerifyUC->>MFARepo: find_active_by_user_and_purpose(user.id, dto.purpose)
    MFARepo-->>VerifyUC: MFAChallenge | None

    alt No existe challenge
        VerifyUC-->>Cliente: MFAChallengeNotFoundException
    else Challenge recuperado
        alt Consumido, expirado o excedido
            VerifyUC-->>Cliente: InvalidOrExpiredMFACodeException / MFAAttemptsExceeded
        else Challenge vigente
            VerifyUC->>Hasher: verify(dto.code, challenge.code_hash)
            Hasher-->>VerifyUC: true | false

            alt Codigo invalido
                VerifyUC->>MFARepo: update(challenge.attempts + 1)
                VerifyUC-->>Cliente: InvalidOrExpiredMFACodeException
            end
        end
    end
end

```

```

        elseCodigo valido
        VerifyUC->>MFARepo: update(consumed_at, attempts)
        VerifyUC->>TokenProvider: generate_access_token(...)
        VerifyUC->>TokenProvider: generate_refresh_token(user.id)
        VerifyUC-->>Cliente: AuthResponseDTO(tokens)
    end
end
end
end
end

```

12. Maqueo entre dominio y adaptadores

Las implementaciones actuales de los puertos son:

Puerto del dominio	Implementacion actual	Tecnologia
UserRepository	DjangoUserRepository	Django ORM
MFARepository	DjangoMFARepository	Django ORM
PasswordHasher	DjangoPasswordHasher	django.contrib.auth.hashers
TokenProvider	JwtTokenProvider	PyJWT
NotificationService	EmailNotificationService	Django email backend

Esto confirma que el dominio se mantiene estable mientras los detalles de persistencia, correo y tokenizacion se cambian en infraestructura.

13. Observaciones de diseno del estado actual

13.1 Dominio anemico

Las entidades son `dataclass` sin metodos de comportamiento. Las invariantes se aplican mayoritariamente desde los casos de uso. Esto simplifica el codigo, pero desplaza reglas del negocio fuera de las entidades.

13.2 Value object pendiente

Email aun no esta implementado. Un objeto de valor ayudaria a encapsular:

- Normalizacion.
- Validacion de formato.
- Igualdad por valor.
- Reglas de identidad del correo.

13.3 Brecha entre login y verificacion MFA

El flujo de `VerifyMFAUseCase` espera un `MFAChallenge` persistido y recuperable por `MFARepository`. Sin embargo, el `LoginUserUseCase` actual envia un codigo fijo por correo y no crea ni persiste el desafio MFA. Desde una perspectiva de documentacion, esto implica que el diseno objetivo del dominio esta mas avanzado que la implementacion actual del login con MFA.

14. Riesgos y consideraciones tecnicas

14.1 Riesgos funcionales

- Si no se persiste el desafio MFA en login, la verificacion MFA no puede completarse correctamente.
- El uso de un codigo fijo en login no cumple con un flujo MFA real.
- La ausencia de metodos de dominio puede facilitar inconsistencias si la logica se dispersa entre casos de uso.

14.2 Controles positivos ya presentes

- Uso de hashes para secretos.
 - Excepciones de dominio bien separadas.
 - Limitacion de intentos MFA.
 - Control de expiracion y consumo de desafios.
 - Inversion de dependencias mediante puertos.
-

15. Recomendaciones de evolucion

15.1 Enriquecer el dominio

Se recomienda mover comportamiento hacia las entidades o servicios de dominio, por ejemplo:

- `User.can_login()`
- `MFAChallenge.is_expired(now)`
- `MFAChallenge.register_failed_attempt()`
- `MFAChallenge.consume(now)`

15.2 Implementar value objects

Prioridad sugerida:

- `Email`
- `HashedSecret`

- MFAPurpose
- MFACHannel

15.3 Cerrar el flujo MFA

El caso de uso de login deberia:

- Generar un codigo aleatorio.
 - Hashearlo.
 - Crear MFACHallenge.
 - Persistirlo mediante MFARepository.
 - Enviar el codigo por NotificationService.
-

16. Resumen ejecutivo

La capa **domain** esta correctamente orientada a arquitectura hexagonal y define con claridad las piezas esenciales del negocio de autenticacion: usuario, desafio MFA, contratos de persistencia y servicios, y errores funcionales. Su principal fortaleza es el desacoplamiento respecto de Django y de la infraestructura. Su principal oportunidad de mejora es incorporar mas comportamiento dentro del dominio y completar la implementacion operativa del flujo MFA para que el modelo y la ejecucion queden alineados.